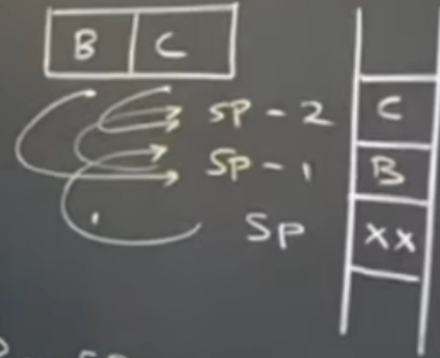


1) Push Rr

Eg: Push B



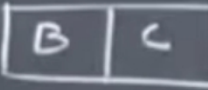
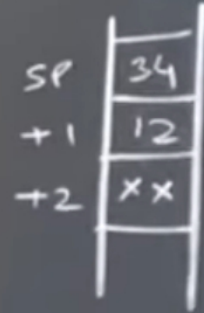
$SP \leftarrow SP - 1$

$[SP] \leftarrow B$

$SP \leftarrow SP - 1$

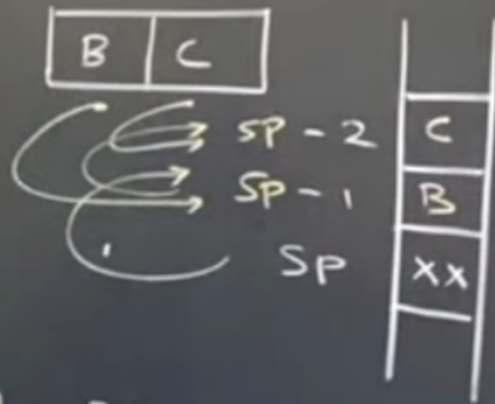
$[SP] \leftarrow C$ //

2) Pop B



1) Push R_r

Eg: Push B



$SP \leftarrow SP - 1$

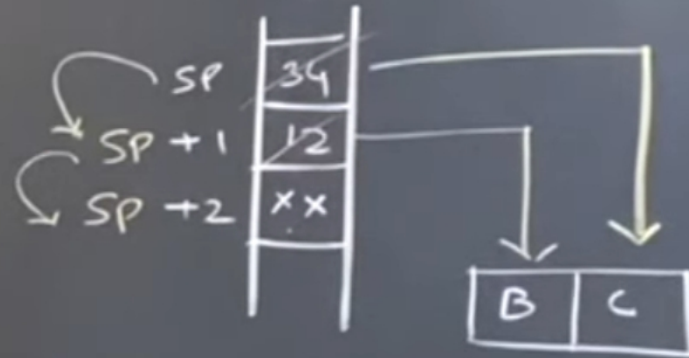
$[SP] \leftarrow B$

$SP \leftarrow SP - 1$

$[SP] \leftarrow C$

//

2) Pop B



$C \leftarrow [SP]$

$SP \leftarrow SP + 1$

$B \leftarrow [SP]$

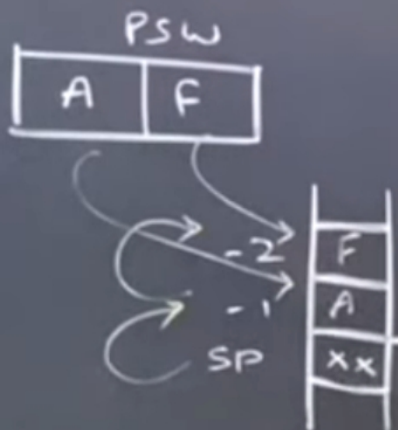
$SP \leftarrow SP + 1$

//

3) Push

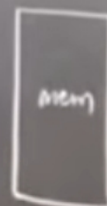
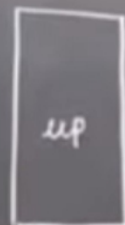
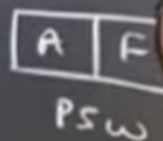
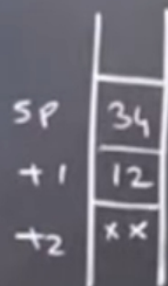


3) Push PSW



$SP \leftarrow SP - 1$
 $[SP] \leftarrow A$
 $SP \leftarrow SP - 1$
 $[SP] \leftarrow F$

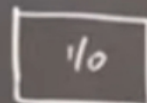
4) Pop PSW



mem address (loc 2's)

00 00H
 ...
 FFFFH

16 bit
 16
 2 = 64K



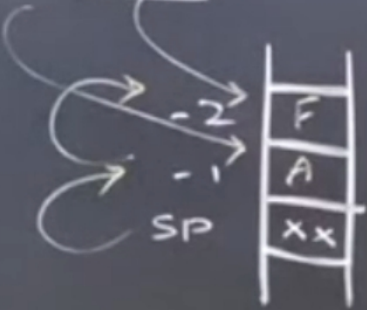
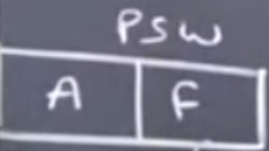
I/O address (ports)

00H
 1
 FFH

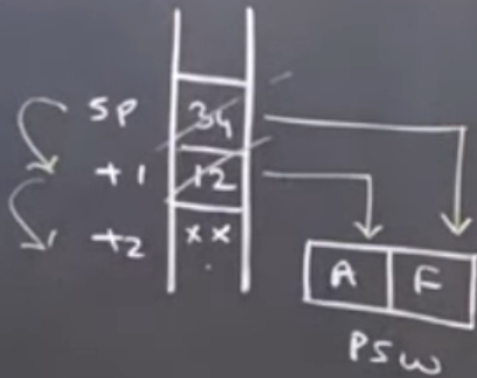
8
 2 = 256

IN
 E:
 OUT
 E:

push PSW

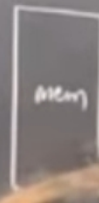
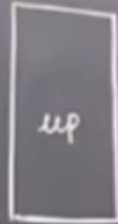


u) Pop PSW

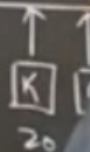
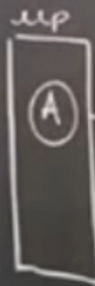


$SP \leftarrow SP - 1$
 $[SP] \leftarrow A$
 $SP \leftarrow SP - 1$
 $[SP] \leftarrow F$

$F \leftarrow [SP]$
 $SP \leftarrow SP + 1$
 $A \leftarrow [SP]$
 $SP \leftarrow SP + 1$



mem address (loc²s)
 0000H
 ...
 FFFFH
 16bit
 16
 2
 = 64K

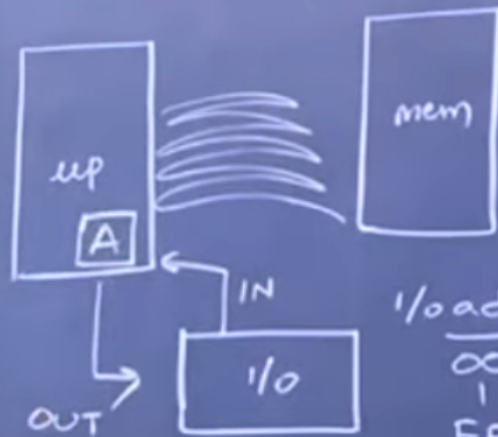
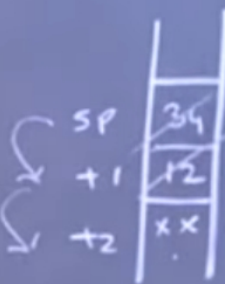


Machine Control

- 1) SIM - Set Int
- 2) RIM - Read Int
- 3) EI - Enable
- 4) DI - Disable
- 5) NOP - No op
- 6) HLT - Halt

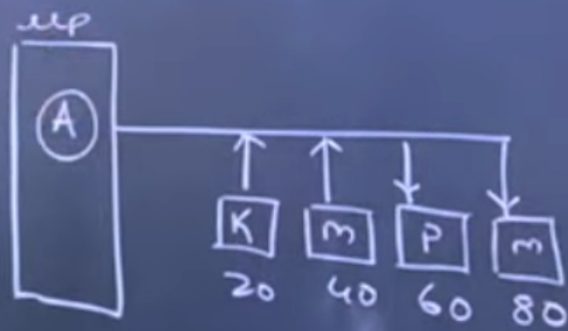
I/O

u) Pop PSW



mem addresses (16 bits)
0000H
⋮
FFFFH
16 bit
 $2^{16} = 64K$

i/o addresses (ports)
00H
⋮
FFH
8 bit
 $2^8 = 256$



✓ IN i/o addr

Eg: IN 20H

$A \leftarrow [20]_{i/o}$

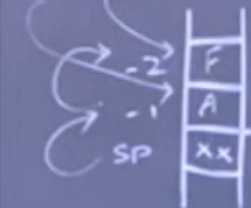
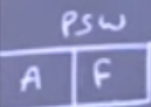
✓ OUT i/o addr

Eg: OUT 80H

$A \rightarrow [80]_{i/o}$

ation.

push PSW



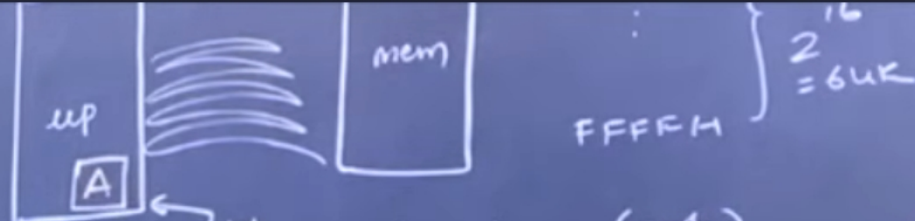
$SP \leftarrow SP - 1$

$[SP] \leftarrow A$

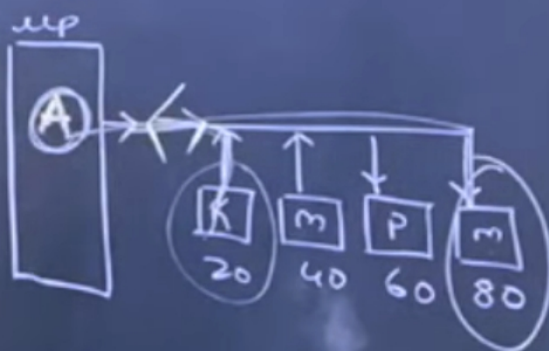
$SP \leftarrow SP - 1$

$[SP] \leftarrow A$

u) Pop PSW



I/O addresses (ports)
00H } 2⁸ = 256
FFH



IN 20H

OUT 80H

✓ IN i/o addr

Ex: IN 20H

$A \leftarrow [20]_{i/o}$

✓ OUT i/o addr

Ex: OUT 80H

$A \rightarrow [80]_{i/o}$

Machine Control

- 1) SIM - Set int mask
- 2) RIM - Read Int mask
- 3) EI - Enable int
- 4) DI - Disable int
- 5) NOP - No operation (Delay)
- 6) HLT - Halt

o add-

20H

- [20] i/o

i/o add-

OT 80H

> [80]

STACK, I/O AND MACHINE CONTROL GROUP

Special Note:

In the explanation of every instruction, I have mentioned its machine cycles and T-states. You will understand this part once you watch the two videos of timing diagrams

STACK INSTRUCTIONS

1) PUSH Rp

It pushes the given register pair into the stack.

Eg: PUSH B ; SP $\leftarrow$ SP - 1
[SP] $\leftarrow$ B
SP $\leftarrow$ SP - 1
[SP] $\leftarrow$ C



Addr. Mode	Flags Affected	Cycles	T-States
Register	None	3	12

Special Notes:

All Stack operations (Push and Pop) are compulsorily **16-bit operations**. We can push register pairs, and not individual 8-bit registers.

Push and Pop only support **Register Addressing Mode**. Hence if we want to push a number like 2000H, we must first move it into a register pair and then push it.

Eg: LXI B, 2000H
Push B

2) POP Rp

It pops the top 2 elements ($2 \times 8\text{-bit} \therefore 16\text{-bit}$) from the stack into the given register pair.

The lower byte comes out first as the higher byte was pushed in first and stack operates in LIFO manner.

Eg: POP B ; C $\leftarrow$ [SP]
SP $\leftarrow$ SP + 1
B $\leftarrow$ [SP]
SP $\leftarrow$ SP + 1

Addr. Mode	Flags Affected	Cycles	T-States
Register	None	3	10

3) PUSH PSW

It pushes the PSW (Program Status Word) into the stack.

The PSW is the combination of the accumulator and the Flag register, accumulator being the higher byte.

∴ **PSW → AF**

PSW can **ONLY** be used in PUSH and POP instructions.

Eg: **PUSH PSW** ; SP ← SP - 1
[SP] ← A
SP ← SP - 1
[SP] ← F

Addr. Mode	Flags Affected	Cycles	T-States
Register	None	3	12



4) POP PSW

It pops the top 2 elements ($2 \times 8\text{-bit}$ ∴ 16-bit) from the stack into the PSW.

Eg: **POP PSW** ; F ← [SP]
SP ← SP + 1
A ← [SP]
SP ← SP + 1

Addr. Mode	Flags Affected	Cycles	T-States
Register	None	3	10

Please Note: There are other instructions like XTHL, SPHL, LXI SP, CALL RET etc which directly or indirectly affect the stack and are already included in various groups above.

Special Note:

PSW is only created to allow push and pop of Accumulator and Flags.
Hence PSW can only be used in Push and Pop instructions.

I/O INSTRUCTIONS

5) IN 8-bit I/O Port address

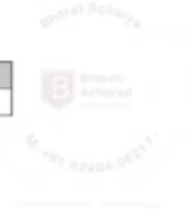
8085 has 256 I/O Ports having 8-bit addresses 00H ... FFH.

This instruction is used to read data from an I/O Port, whose address is given in the instruction.

This data can be read into the Accumulator ONLY.

Eg: **IN 80** ; A ← [80]_{I/O}

Addr. Mode	Flags Affected	Cycles	T-States
Direct	None	3	10



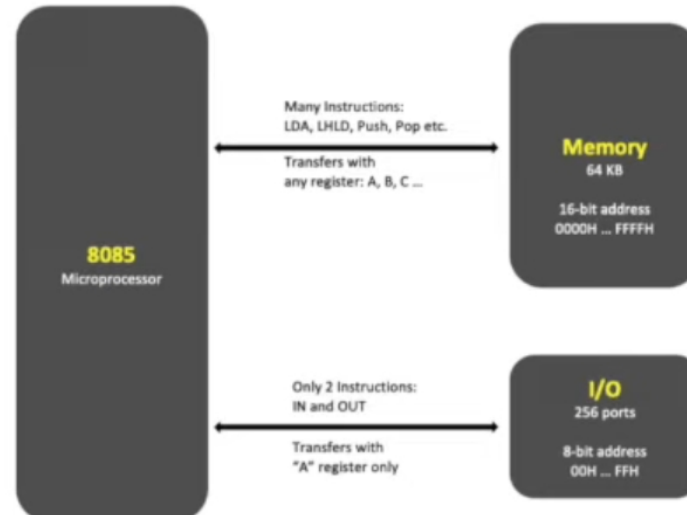
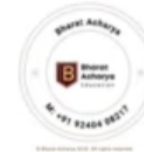
6) OUT 8-bit I/O Port address

This instruction is used to send data from the accumulator to an I/O Port, whose address is in the instruction.

This data can be sent from the Accumulator ONLY.

Eg: **OUT 80**; [80]_{I/O} ← A

Addr. Mode	Flags Affected	Cycles	T-States
Direct	None	3	10



MACHINE CONTROL INSTRUCTIONS

7) SIM (Set Interrupt Mask)

This instruction is used to set the interrupt masking pattern for the μP .

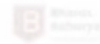
The appropriate bit pattern is **loaded into** the accumulator and then this instruction is executed.

It is basically used to **mask/unmask** the **interrupts** except TRAP and INTR.

It can also be used to send a 'bit' out through the serial out pin **SID**.

Eg: SIM ; μP accepts the masking pattern through the accumulator.

Addr. Mode	Flags Affected	Cycles	T-States
Implied	None	1	4



8) RIM (Read Interrupt Mask)

This instruction is used to read the interrupt masking pattern for the μP .

After executing this instruction, the μP loads the bit pattern **into** the accumulator.

It can also be used to receive a 'bit' out through the serial in pin **SID**.

Eg: RIM ; μP loads the masking pattern into the accumulator.

Addr. Mode	Flags Affected	Cycles	T-States
Implied	None	1	4

Please Note: For the bit patterns of SIM and RIM instruction, please refer the chapter on Interrupts.

9) EI (Enable Interrupts)

This instruction is used to enable the interrupts in the μP .

This instruction sets the INTE flip-flop (Interrupt Enable Flip-Flop).

This instruction effects all the interrupts except TRAP.

Eg: EI ; $INTE_{FF} \leftarrow 1$

Addr. Mode	Flags Affected	Cycles	T-States
Implied	None	1	4

10) DI (Disable Interrupts)

This instruction is used to disable the interrupts in the μP .

This instruction resets the INTE flip-flop.

This instruction effects all the interrupts except TRAP.

Eg: DI ; $INTE_{FF} \leftarrow 0$

Addr. Mode	Flags Affected	Cycles	T-States
Implied	None	1	4

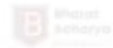


11) NOP (No Operation)

This instruction performs no operation, but consumes time of the μP . It is the simplest method of causing a software delay.

Eg: NOP ; -----

Addr. Mode	Flags Affected	Cycles	T-States
Implied	None	1	4



12) HLT (Halt)

This instruction signifies the end of the program.

It causes the μP to stop fetching any further instruction, hence program execution is stopped.

It makes the Halt Flip Flop inside the $\mu P = 1$.

μP checks the Halt Flip Flop in the beginning (1st T-State) of the next Machine Cycle and Stops all operations.

Eg: HLT ; Halt Flip-Flop $\leftarrow 1$

Addr. Mode	Flags Affected	Cycles	T-States
Implied	None	1 + 1T	5

Bharat Acharya Education

Learn...

8085 | 8086 | 80386 | Pentium |

8051 | ARM7 | COA



© Bharat Acharya 2025. All rights reserved.